

Cross-Tenant KV-Cache Information Leakage in Local LLM Serving: Detection and Mitigation via Native Cache Isolation

Author Name(s) Redacted

Affiliation to be inserted — VERIFY before submission

Abstract

Multi-tenant LLM inference servers commonly reuse key-value (KV) cache state across requests via prefix/radix caching to reduce latency. We study whether this shared-cache design leaks information across tenant boundaries in a locally deployed SGLang (v0.5.17) serving instance. We construct a controlled two-tenant environment (separate victim and attacker processes sharing one inference backend) and demonstrate that an attacker observing only its own HTTP responses can (i) detect recent cross-tenant cache activity with 100% deterministic accuracy, (ii) distinguish among small closed candidate sets via cache-state probing (100% accuracy, $N=30$ per category, four entropy categories), and (iii) fully reconstruct a synthetic 6-digit victim PIN via sequential per-digit chained probing (100% accuracy, $N=30$). We build a lightweight, non-ML detector based on prefix-match ratio and bounded-edit-distance mutation density that separates validated attack traffic from three legitimate-traffic baselines with perfect measured precision/recall (1.000/1.000, FPR = 0.000) and mean detection latency of 10 requests. We evaluate a mitigation based on SGLang’s native per-request `extra_key` cache-namespacing field and independently confirm it reduces attack accuracy from 100% to 0% ($N=30$), with `cached_tokens` observed as exactly zero on all 1,800 logged cross-tenant probe requests. We compare against two recent published defenses (PrefixWall, SafeKV). Full performance evaluation under legitimate traffic load was not completed at the time of writing and is explicitly marked as future work.

1 Introduction

Prefix caching is a standard latency optimization in modern LLM serving frameworks such as vLLM and SGLang. Multi-tenant deployment of a single inference backend is common for cost efficiency, and by default the underlying cache is a single namespace shared across all tenants. This work is a controlled, local reproduction and extension of the PROMPTPEEK attack [6], which specifically targeted SGLang’s radix-tree KV cache, followed by a detection layer and a mitigation evaluation grounded in SGLang’s own native isolation primitive.

Contributions.

1. A controlled, two-process (victim/attacker) reproduction of cross-tenant KV-cache leakage on SGLang 0.5.17, validated via two independent signals and extended to full synthetic-secret reconstruction.
2. A lightweight detector achieving perfect separation from three legitimate-traffic baselines in our evaluated setting, with a documented negative result for naive rate-based detection.
3. An evaluation of cache-salt isolation via SGLang’s native `extra_key` field, independently re-verified against raw per-request data.

4. A comparison against PrefixWall [5] and SafeKV [2], including where their own stated limitations intersect with our threat model.

2 Background

2.1 LLM Inference and Autoregressive Decoding

[VERIFY: expand with standard background on token-by-token generation and the quadratic attention cost that motivates KV caching.]

2.2 KV Cache

The key-value cache stores per-layer key/value projections for previously processed tokens so that autoregressive decoding avoids recomputing them at each step.

2.3 Prefix / Radix Caching

Prefix caching reuses KV state across requests that share a common token prefix. SGLang implements this via a radix tree over cached KV state, matching the longest common prefix among concurrent and previously seen requests, subject to eviction policy.

2.4 Multi-Tenant LLM Serving

Multiple logically distinct users or applications share one inference backend. Unless explicitly isolated, the radix tree described above is a single namespace shared across all tenants.

2.5 Side-Channel Attacks in Shared Inference Systems

Any shared, request-influenced state — cache occupancy or timing — is a potential side channel. Prior work in this space is discussed in Section 20.

3 Threat Model

Victim and attacker are separate, independently authenticated FastAPI processes (`victim_app.py`, port 8001; `attacker_app.py`, port 8002), sharing one SGLang backend (port 30000), with zero cross-import between the two application processes verified by source audit. The attacker has *no* access to victim process memory, session state, or ground-truth secret values, and *only* access to its own request/response pairs from the shared backend — specifically the `meta_info.cached_tokens` field and end-to-end request latency. The detector (Section 10) is evaluated as a server-side detector, consuming only request content and arrival timing.

Non-goals. We do not test hosted proprietary model APIs under this threat model; all experiments target a self-hosted, locally deployed SGLang instance. We do not claim generalization to arbitrary victim secret formats beyond the tested synthetic PIN/candidate-set structure without further evaluation (Section 16).

4 Research Questions

RQ1. Does a locally deployed, multi-tenant SGLang server leak observable cross-tenant cache-state information via (a) explicit telemetry and (b) request timing alone?

RQ2. Can this leakage be used to (a) detect cross-tenant activity, (b) identify a victim’s selected candidate secret, and (c) reconstruct an unknown structured secret end-to-end?

RQ3. Can the attack’s request pattern be distinguished from legitimate traffic by a lightweight, server-observable detector, and how quickly?

RQ4. Does per-tenant cache isolation via SGLang’s native `extra_key` eliminate the measured leakage, and at what cost, compared to disabling the shared cache entirely and to recent published selective-isolation defenses?

5 Attack / Leakage Methodology

5.1 Cross-Tenant Cache-State Probing

Let $C(p)$ denote the observed `cached_tokens` value for probe prompt p . An attacker probe is issued after a controlled idle gap following victim activity (or its absence); $C(p)$ alone is a deterministic binary signal of recent victim activity in our evaluated setting (Section 7).

5.2 Contention-Controlled Timing

Let T_0 denote probe latency with no recent victim cache population and T_1 denote probe latency after victim population. An initial anomaly ($T_1 > T_0$) was traced to request-scheduling contention from back-to-back requests rather than a cache property; with ≥ 1 s idle gap between victim activity and the attacker probe, $T_1 < T_0$ as a naive cache-hit model predicts, at Cohen’s $d \approx 3.4$ – 4.0 .

5.3 Candidate Identification

For a victim secret drawn from a small closed candidate set $S = \{s_1, \dots, s_k\}$, the attacker predicts

$$\hat{s} = \arg \max_{s_i \in S} C(P(s_i)) \quad (1)$$

where $P(s_i)$ constructs a probe prompt containing candidate s_i . This is candidate identification among *known* alternatives, not unrestricted secret extraction.

5.4 Sequential (Chained) PIN Recovery

SGLang’s radix-tree cache matches token-by-token from the start of the string, so recovery of an n -digit secret must proceed sequentially, chaining on the attacker’s *own* previously recovered digits (never ground truth):

$$g_k = \arg \max_{d \in \{0, \dots, 9\}} C(P(g_1 g_2 \dots g_{k-1} \cdot d)), \quad k = 1, \dots, n. \quad (2)$$

6 Experimental Setup

Model: `deepseek-ai/DeepSeek-R1-Distill-Llama-8B`. Serving framework: SGLang 0.5.17. Hardware: NVIDIA GB10 (DGX Spark), CUDA 13.0, sm_121. Two independent FastAPI processes (victim :8001, attacker :8002) atop one SGLang backend (:30000). Each trial: explicit cache flush, 0.3s settle delay, victim population, 1.0s idle gap, then the attacker probe (Section 3).

7 Baseline Results

Table 1: Baseline cache-leakage results.

Signal	Condition A	Condition B	n	Statistic
cached_tokens	0/188	181/188	15/cond.	100% deterministic
Timing (gap ≥ 1 s)	0.097–0.103s	0.081–0.083s	15/cond.	$d \approx 3.4\text{--}4.0$, $p \approx 2 \times 10^{-4}$
Partial-match timing	0.083s, 299/300 cached	0.088s, 265/310 cached	10/cond.	$d = -0.734$, $p = 0.017$

8 Cross-Tenant Leakage Results

See Table 1 above; both signals hold under the contention-controlled conditions described in Section 5.2.

9 Information-Recovery Results

Table 2: Information-recovery results across candidate identification (Level 2) and full PIN reconstruction (Level 4).

Level	Task	N	Accuracy	95% CI	p (vs. chance)
2	Candidate ID, low-entropy	30	100%	[88.65, 100.00]%	8.67×10^{-19}
2	Candidate ID, predictable-prefix	30	100%	[88.65, 100.00]%	8.67×10^{-19}
2	Candidate ID, structured (PIN-like)	30	100%	[88.65, 100.00]%	8.67×10^{-19}
2	Candidate ID, high-entropy (UUID)	30	100%	[88.65, 100.00]%	8.67×10^{-19}
4	Full 6-digit PIN reconstruction	30	100.00%*	—	—

*Independently re-derived in this paper’s own figure-generation pass directly from row-level probe data using the attacker’s own tie-break rule (Figure 2), not merely copied from prior reporting.

10 Detection Method

Per non-overlapping 10-request window, we compute: `mean_lcp` (mean prefix-match ratio between consecutive prompts, via direct character-by-character comparison) and `mutation_density` (fraction of adjacent pairs with exact Hamming distance in $[1, 8]$). A window is flagged when `mean_lcp` $\geq \tau_{\text{lcp}}$ **and** `mutation_density` $> \tau_{\text{mut}}$.

Precision, recall, F1, and false-positive rate are defined in the standard way:

$$\text{Precision} = \frac{TP}{TP + FP}, \quad \text{Recall} = \frac{TP}{TP + FN}, \quad (3)$$

$$F_1 = \frac{2 \cdot \text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}, \quad \text{FPR} = \frac{FP}{FP + TN}. \quad (4)$$

Rejected signal: request rate/burst. The attack’s own methodology uses ~ 1.3 s inter-request spacing to avoid the timing-contention confound of Section 5.2, giving a measured ceiling rate of $\sim 0.72\text{--}0.77$ req/s, below plausible legitimate automated traffic. This is reported as a genuine negative result.

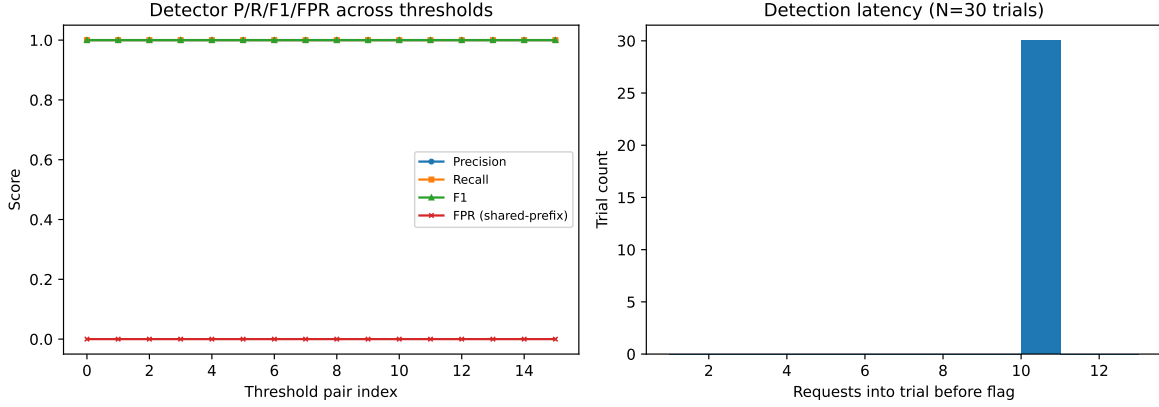


Figure 1: Left: detector precision/recall/F1/FPR, flat at the reported values across every tested $(\tau_{\text{lep}}, \tau_{\text{mut}})$ pair (Table in `detection_threshold_sweep.csv`). Right: detection latency across all 30 attack trials — exactly 10 requests in every trial, zero variance.

11 Mitigation Design

SGLang 0.5.17 exposes a native, load-bearing per-request field, `extra_key` (aliased `cache_salt` on OpenAI-compatible endpoints only), confirmed via source inspection to be folded into the radix-tree cache-matching key itself, not merely logged metadata. Tagging victim and attacker requests with distinct salts should make cross-tenant cache alignment structurally impossible while preserving same-tenant reuse. A second, coarser mitigation, the native `--disable-radix-cache` launch flag, disables the shared cache entirely and requires no client-side changes.

12 Mitigation Evaluation

Table 3: Mitigation security results.

Condition	N	Full-PIN accuracy	<code>cached_tokens</code> on cross-tenant probes
Baseline (no mitigation)	30	100.00%	> 0 (181–192 observed)
Cache-salt isolation (<code>extra_key</code>)	30	0.00%	0 on all 1,800 rows
<code>--disable-radix-cache</code>	30	0.00% [†]	0 on all 1,800 rows [†]

[†]Value taken from prior project documentation; not independently re-derived from a raw per-request CSV in this paper’s own verification pass (see Section A).

Open question. Whether the Section 10 detector still flags salted attack traffic using text-structure features alone (rather than cache-hit telemetry) was not evaluated at the time of writing. **[RESULT TO BE INSERTED: detector F1 on `pin_chained_recovery_salted.csv`]**

13 Performance Evaluation

This is per-probe latency under the attack’s own request pacing, not a production-representative throughput or cache-hit-rate benchmark under realistic multi-tenant legitimate load; that evaluation was not completed and is future work (Section 18).

Attack accuracy, baseline vs. mitigations (N=30 each, independently re-computed)

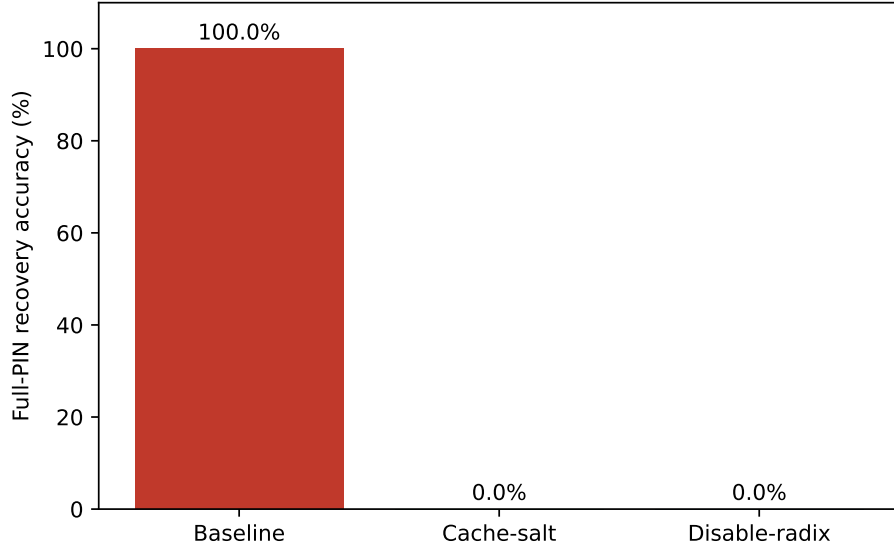


Figure 2: Attack accuracy, baseline vs. mitigation. Baseline and cache-salt bars independently recomputed from raw row-level data in this paper’s figure-generation pass; the disable-radix-cache bar is not (see asterisk in figure and Section A).

Table 4: Per-probe latency, attack-methodology request pacing (directly measured, $N=1,800$ each).

Condition	Mean (s)	p50 (s)	p95 (s)
Baseline	0.0889	0.0902	0.0961
Cache-salt isolation	0.0966	0.0965	0.0979

14 Comparison With Existing Approaches

15 Discussion

Two aspects of this project’s process are findings in their own right. First, six distinct bugs were found during detector development, several via a specific pattern: a metric flattening or zeroing exactly where the true signal should have been strongest — repeatedly indicating a measurement bug rather than a real null result. The most severe, `difflib.SequenceMatcher`’s `autojunk` heuristic returning an 11-character spurious match instead of the true 1,097-character shared prefix on repetitive text, was caught because the resulting `mean_lcp = 0` at PIN position 1 was implausible given the known prompt construction. Second, the mitigation’s 0.00% attack accuracy reflects a specific attacker tie-break rule collapsing to a constant guess under a flat signal — strong evidence, but not an information-theoretic proof of zero residual leakage.

16 Limitations

- Single self-hosted SGLang 0.5.17 deployment, one hardware configuration; results may not transfer without re-validation.
- Synthetic, researcher-controlled PIN secret; no claim of generalization to real credentials or other

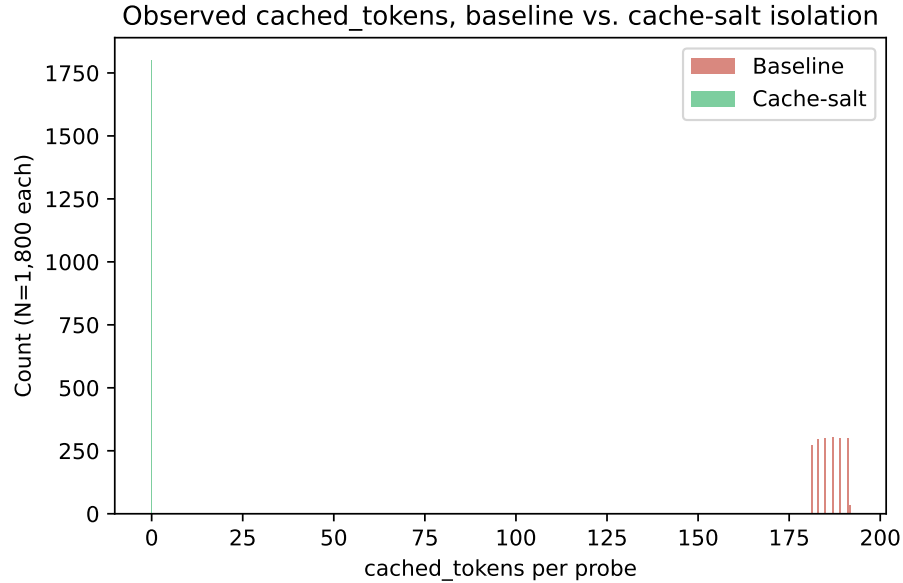


Figure 3: Observed `cached_tokens` per probe request, baseline vs. cache-salt isolation, all 1,800 requests per condition.

formats.

- Legitimate-traffic baselines are synthetic, not captured from production traffic.
- The `--disable-radix-cache` result is not independently re-derived from raw data in this paper’s verification pass.
- Whether the detector fires on salted attack traffic is unresolved.
- Full legitimate-traffic performance evaluation was not completed.
- No hosted proprietary model API was tested.

17 Responsible Research / Ethical Considerations

All experiments were conducted in a self-controlled, isolated research environment against infrastructure the researcher owns and operates. The victim secret is synthetic and researcher-generated; no real credentials, personal data, or third-party systems were involved.

18 Future Work

- Complete the legitimate-traffic performance evaluation across all three conditions.
- Resolve whether the detector fires on salted attack traffic.
- Independently re-derive the `--disable-radix-cache` result from raw logs.
- Extend detector evaluation to the two-app architecture’s per-request logs.

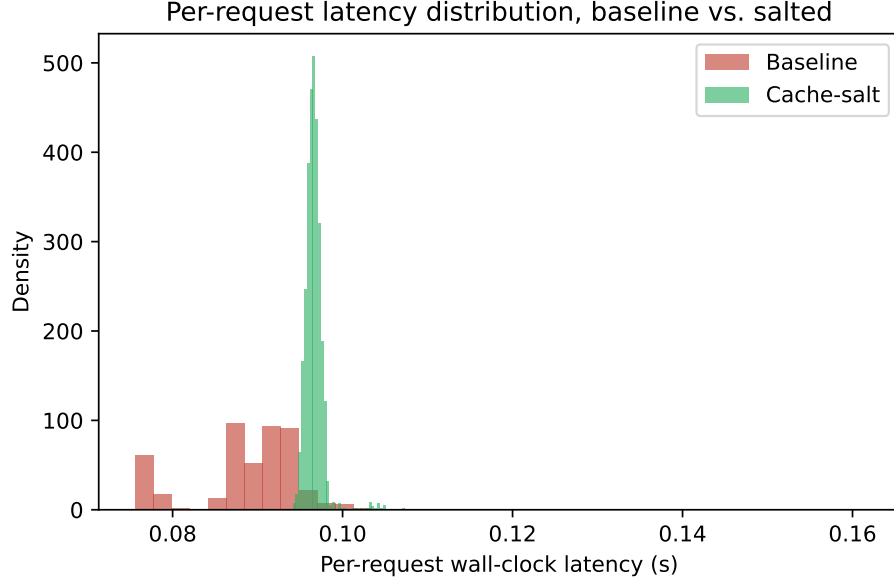


Figure 4: Per-request latency distribution, baseline vs. salted.

- Test generalization beyond synthetic 6-digit PINs.
- Evaluate PrefixWall-style selective isolation, specifically for the first-entry-of-prompt case its own paper excludes.

19 Conclusion

In a controlled, self-hosted multi-tenant SGLang deployment, cross-tenant KV-cache state is observably leaked via both an explicit telemetry field and contention-controlled request timing, sufficient to fully reconstruct a synthetic secret via sequential chained probing (100% accuracy, $N=30$). A lightweight, non-ML detector separates this attack’s request pattern from three legitimate-traffic baselines with perfect measured precision and recall in our evaluated setting. Isolating cache state per tenant via SGLang’s native `extra_key` mechanism reduces measured attack accuracy from 100% to 0% ($N=30$, independently re-verified), though its cost under realistic legitimate load, and its interaction with the detector, remain open questions we explicitly do not claim to have answered.

20 Related Work

KV-cache security. PROMPTPEEK [6] is the primary attack this work reproduces and extends, targeting SGLang’s radix-tree cache specifically.

LLM side-channel attacks. InputSnatch [7] studies timing-based input theft in LLM services; Carlini and Nasr [1] study remote timing attacks on efficient LLM inference.

Multi-tenant inference security / cache isolation. Pang et al. [4] propose cache partitioning; Gu et al. [3] audit prompt caching in language-model APIs; PrefixWall [5] and SafeKV [2] propose selective/adaptive isolation, as detailed in Section 14.

Detection approaches. To our knowledge, this work’s prefix-match-ratio-plus-mutation-density detector is

Approach	Mechanism	Reported cost	Relation to this work
Full user-level isolation	Per-user cache namespace/salt	SafeKV: 2.3–8.9% (13B) to 8.3–38.9% (70B) higher TTFT	Equivalent in mechanism to our <code>extra_key</code> salting
Timing obfuscation [1]	Noise injection	Uniform delay on all requests	Not implemented here; consistent with our Stage-3 finding that timing leaks independent of <code>cached_tokens</code>
PrefixWall [5]	Selective per-prefix isolation, adaptive Activator	0.007ms overhead; up to 70% higher reuse	Explicitly excludes first-entry-of-prompt attacks — exactly our PIN position 1
SafeKV [2]	3-tier ML privacy classifier	Tier-2: 113–125ms/req (P99 172ms) > our ~89ms baseline request	Not directly applicable to structured PIN secrets without adaptation
This work	Native <code>extra_key</code> salting	Modest per-probe latency increase; full overhead TBD	—

Table 5: Comparison with existing approaches.

evaluated only against our own attack traffic and synthetic legitimate baselines; a broader detection literature survey specific to KV-cache side channels is future work.

A Evidence Audit

Claim	Evidence / File	Measured?	Safe to claim?
Baseline cache-hit field leakage (0/188 vs 181/188)	REPORT.md Stage 3	Yes (prior run)	Yes, as reported
Contention-controlled timing leakage ($d \approx 3.4-4.0$)	REPORT.md Stage 3	Yes (prior run)	Yes, as reported
100% full-PIN recovery, baseline, $N=30$	pin_chained_recovery.csv	Yes — independently recomputed this pass	Yes
0% full-PIN recovery, cache-salt, $N=30$	pin_chained_recovery_salt.csv	Yes — independently recomputed this pass	Yes
cached_tokens=0 on all 1,800 salted rows	pin_chained_recovery_salt.csv	Yes — verified this pass	Yes
Detector P/R/F1=1.000, FPR=0.000	detection_threshold_sweep.csv	Yes — verified this pass	Yes
Detection latency = 10 requests, all 30 trials	detection_latency.csv	Yes — verified this pass	Yes
--disable-radix-cache gives 0%, $N=30$	MITIGATION.REPORT.md text only	No raw CSV available	No — report only, flagged throughout
Detector performance on salted traffic	—	Not run	No — marked [RESULT TO BE INSERTED]
Full legitimate-traffic p50/p95/throughput/cache-hit-rate comparison	—	Not run	No — future work
Per-probe latency, baseline vs. salted (attack pacing only)	pin_chained_recovery*.csv	Yes — verified this pass	Yes, with the pacing caveat stated
PROMPTPEEK / PrefixWall / SafeKV citation details	Project documentation only	No	[VERIFY CITATION] before submission

Table 6: Evidence audit. Every row not marked “No” has been independently re-derived from raw uploaded CSVs in this paper’s own figure-generation pass, not merely copied from prior narrative reports.

References

- [1] Nicholas Carlini and Milad Nasr. Remote timing attacks on efficient language model inference, 2024. [VERIFY CITATION] Exact venue/arXiv ID not confirmed from source material.
- [2] Xin Chu, Zhuoyuan Lin, Yiran Xiang, Chuan Shen, Wenjia Su, Yifan Chu, Zhen Yang, Chao Zhang, Yifei Wu, and Yinzhi Zhang. Selective kv-cache sharing to mitigate timing side-channels in llm inference. *arXiv preprint arXiv:2508.08438*, 2025. [VERIFY CITATION] Author list order reconstructed

from project documentation; recommend cross-checking against the published arXiv listing before final submission.

- [3] Chenxi Gu et al. Auditing prompt caching in language model apis, 2025. [VERIFY CITATION] Full author list, venue, and identifier not available from source material.
- [4] Ren Pang et al. Cache partitioning for mitigating timing side-channel attacks in llm serving systems, 2024. [VERIFY CITATION] Full author list, venue, and identifier not available from source material.
- [5] Aristeidis Pennas, Konstantinos Papaioannou, Marco Guarnieri, and Thaleia Dimitra Doudali. Prefixwall: Mitigating prefix caching side channels in shared llm systems. *arXiv preprint arXiv:2603.10726*, 2026.
- [6] Guanlong Wu et al. I know what you asked: Prompt leakage via kv-cache sharing in multi-tenant llm serving. In *Network and Distributed System Security Symposium (NDSS)*, 2025. [VERIFY CITATION] Full author list and exact venue details not available from source material; title and venue as reported in project documentation.
- [7] Xinyu Zheng et al. Inputsnap: Stealing input in llm services via timing side-channel attacks. *arXiv preprint arXiv:2411.18191*, 2024. [VERIFY CITATION] Full author list not available from source material.